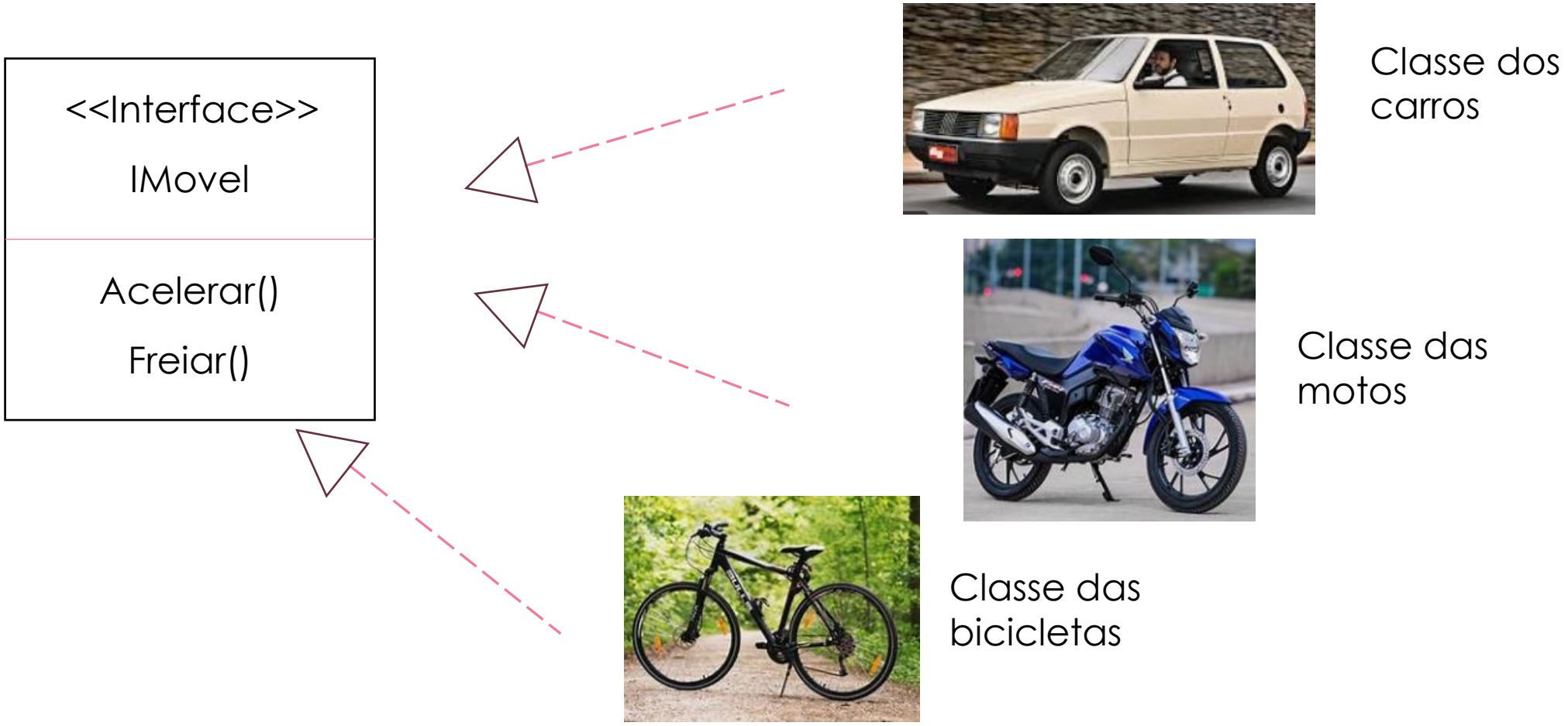
Spring Boot

Interface Collection

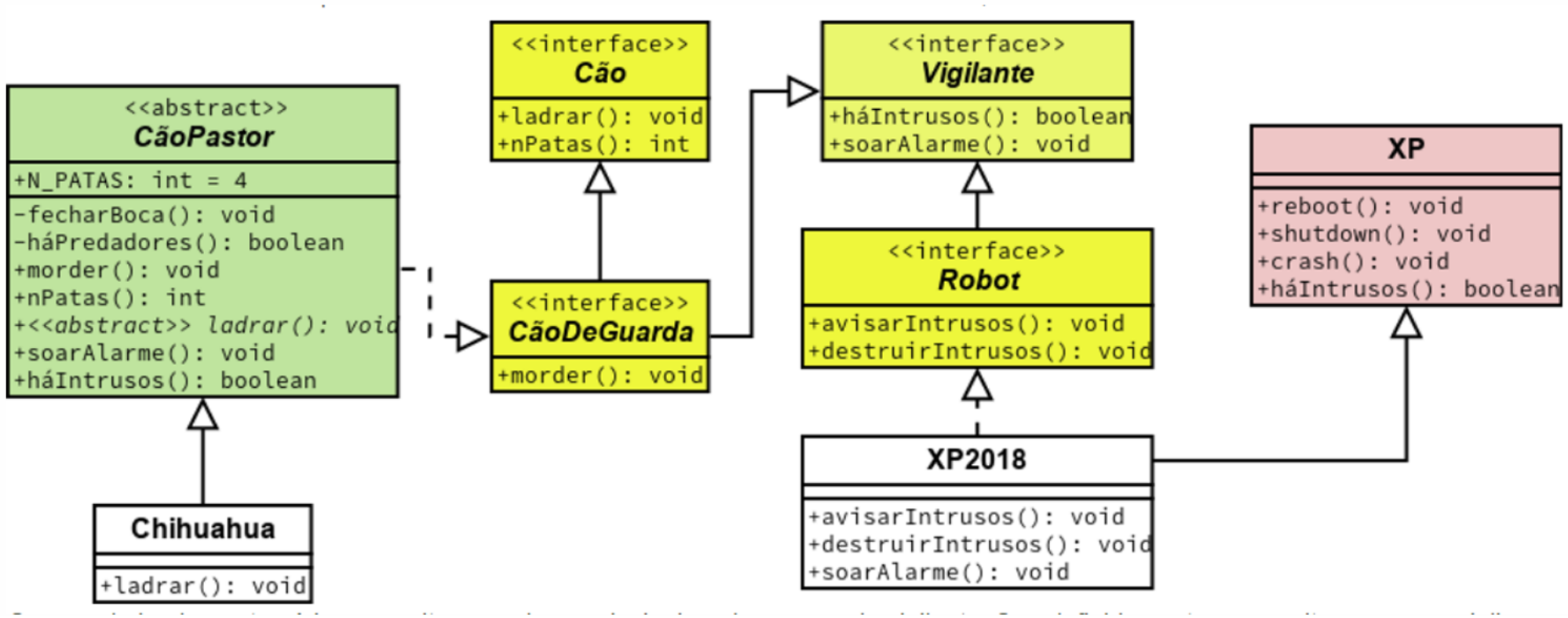
PAULO R T CÂNDIDO

Interface

Uma **Interface** em Java funciona como um "contrato". Ela define **o que** uma classe deve fazer, mas não **como** ela deve fazer. É o pilar fundamental para criar sistemas flexíveis e desacoplados.



Interface


<<abstract>>
CãoPastor
+N_PATAS: int = 4
-fecharBoca(): void
-háPredadores(): boolean
+morder(): void
+nPatas(): int
+<<abstract>> ladrar(): void
+soarAlarme(): void
+háIntrusos(): boolean
<<interface>>
Cão
+ladrar(): void
+nPatas(): int
<<interface>>
CãoDeGuarda
+morder(): void
<<interface>>
Vigilante
+háIntrusos(): boolean
+soarAlarme(): void
<<interface>>
Robot
+avisarIntrusos(): void
+destruirIntrusos(): void
XP
+reboot(): void
+shutdown(): void
+crash(): void
+háIntrusos(): boolean
Chihuahua
+ladrar(): void
XP2018
+avisarIntrusos(): void
+destruirIntrusos(): void
+soarAlarme(): void

Interface – exemplo em código

```
// Interface
interface MetodoPagamento {
    void processarPagamento(double valor); // Método abstrato (sem corpo)
    void exibirRecibo();
}

// Classe que implementa a interface
class CartaoCredito implements MetodoPagamento {

    public void processarPagamento(double valor) {
        System.out.println("Processando pagamento de R$" + valor + " via Cartão de Crédito...");
        System.out.println("Validando limite e transação...");
    }

    public void exibirRecibo() {
        System.out.println("Recibo enviado para o e-mail do cliente.");
    }
}
```

Collection

O ecossistema de **Java Collections Framework (JCF)** é o coração da manipulação de dados em Java. Java oferece uma arquitetura pronta, performática e padronizada para criação de arrays dinâmicos ou estruturas de busca.

O JCF é baseado em interfaces. Isso permite que você escreva código genérico: você declara uma `List` (interface), mas decide no momento da criação se ela será um `ArrayList` ou um `LinkedList` (implementações).

```
List<String> listaArray = new ArrayList<>();
listaArray.add("João");
```

```
List<String> listaLigada = new LinkedList<>();
listaLigada.add("Maria");
```

Collection

As "Quatro Grandes" Interfaces:

•**List:** Uma coleção ordenada que permite elementos duplicados. Você acessa os itens pela posição (índice).

•**Set:** Não permite elementos duplicados. Ideal para conjuntos matemáticos ou garantir unicidade.

•**Queue/Deque:** Focadas na ordem de processamento (Fila: primeiro a entrar, primeiro a sair).

•**Map:** Não herda de `Collection`, mas faz parte do framework. Armazena pares de **Chave e Valor**.

Collection

Interface	Implementação	Característica Principal	Melhor caso de uso
List	ArrayList	Baseada em array redimensionável.	Acesso rápido via índice ($O(1)$).
List	LinkedList	Lista duplamente ligada.	Inserções/remoções frequentes no início/fim.
Set	HashSet	Usa Tabela Hash. Alta performance.	Quando a ordem não importa e não quer duplicatas.
Set	TreeSet	Elementos ordenados (Árvore).	Quando você precisa dos dados sempre classificados.
Map	HashMap	Armazena Chave -> Valor.	Buscas rápidas por uma chave única.
Map	TreeMap	Chaves ordenadas.	Mapas que precisam manter uma ordem específica das chaves.

Collection

Não confunda a interface `Collection` (singular) com a classe utilitária `Collections` (plural). Esta última possui métodos estáticos poderosos:

- `Collections.sort(lista)`: Ordena a lista.
- `Collections.reverse(lista)`: Inverte a ordem.
- `Collections.shuffle(lista)`: Embaralha os elementos.
- `Collections.unmodifiableList(lista)`: Cria uma lista que não pode ser alterada.